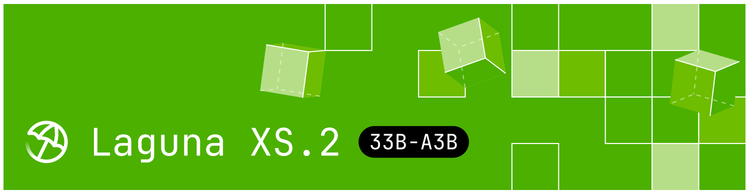
poolside/Laguna-XS.2



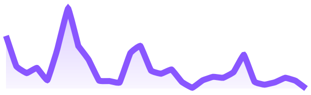
Get an API key · **Release blog post** · **Technical report**

Laguna XS.2

Laguna XS.2 is a 33B total parameter Mixture-of-Experts model with 3B activated parameters per token designed for agentic coding and long-horizon work on a local machine. It uses Sliding Window Attention with per-head gating in 30 out of 40 layers for fast inference and low KV cache requirements.

> For more details on how we trained this model, including on data automixing and async off-policy agent RL, check out our release blog post and technical report.

Highlights



- **Mixed SWA and global attention layout**: Laguna XS.2 uses sigmoid gating with per-layer rotary scales, enabling mixed SWA (Sliding Window Attention) and global attention layers in a 3:1 ratio (across 40 total layers)
- **KV cache in FP8**: KV cache quantized to FP8, reducing memory per token
- **Native reasoning support**: Interleaved thinking between tool calls with support for enabling and disabling thinking per-request
- **Local-ready**: At 33B total parameters and 3B activated, Laguna XS.2 is compact enough to run on a Mac with 36 GB of RAM. Available on Ollama
- **Apache 2.0 license**: Use and modify freely for commercial and non-commercial purposes

---

Model overview

- Training: pre-training, post-training and reinforcement learning stages
- Number of parameters: 33B total with 3B activated per token
- Optimizer: Muon
- Layers: 40 layers (10 layers with global attention, 30 layers with sliding window attention)
- Experts: 256 experts with 1 shared expert
- Sliding Window: 512 tokens
- Modality: text-to-text

Collection including poolside/Lag...

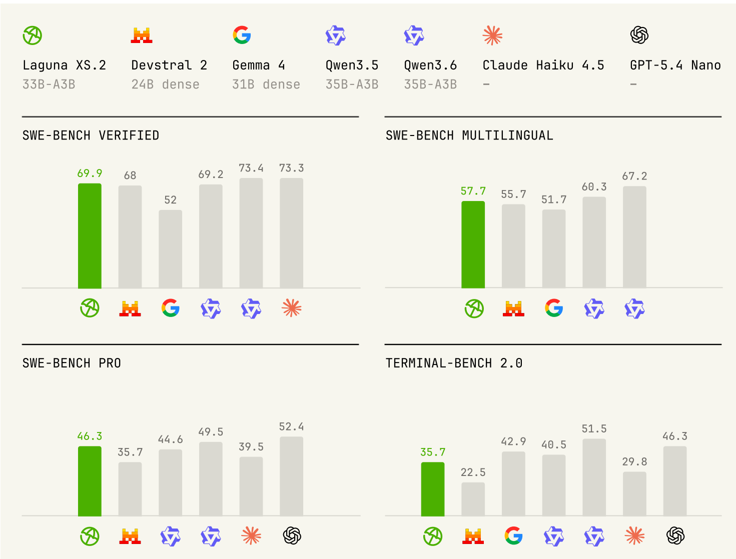
Laguna XS.2 Collection
Designed for ... • 5 items • U... • 28

Evaluation results

ScaleAI/SWE-bench_Pro · S...	46.3
SWE-bench/SWE-bench_Veri...	69.9
harborframework/terminal-...	35.7

- Context window: 262,144 tokens
- Reasoning support: interleaved thinking with preserved thinking

Benchmark results



Model	Size (total params.)	SWE-bench Verified	SWE-bench Multilingual	SWE-bench Pro (Public Datase
Laguna XS.2	33B	69.9%	57.7%	46.3%
Devstral Small 2	24B dense	68.0%	55.7%	-
Gemma 4 31B IT	31B dense	52.0%	51.7%	35.7%
Qwen3.5-35B-A3B	35B	69.2%	60.3%	44.6%
Qwen3.6-35B-A3B	35B	73.4%	67.2%	49.5%

Model	Size (total params.)	SWE-bench Verified	SWE-bench Multilingual	SWE-bench Pro (Public Datase
Claude Haiku 4.5	-	73.3%	-	39.5%
GPT-5.4 Nano	-	-	-	52.4%

We used the highest publicly-referenced scores for all comparison models across each benchmark. In almost all cases these were official scores published in release blog posts or equivalent, with the exception of Gemma 4 31B IT where the highest published scores were *reported by the Qwen team* *and Claude Haiku 4.5 where the highest published (verified) scores for SWE-bench Pro and Terminal-Bench 2.0 are from their respective official leaderboards.*

▶ Expand for benchmarking methodology

Usage

Laguna XS.2 has launch-day support in vLLM and Transformers, and TRT-LLM thanks to the support of the team at NVIDIA.

The fastest way to get started is with our API, directly or using OpenRouter.

> We are providing free access for a limited time to Laguna XS.2, and our larger 225B model, Laguna M.1, on our API. You can create an API key on our Platform.

pool

pool is a lightweight terminal-based coding agent and a dual Agent Client Protocol client-server.

Download and install for macOS and Linux:

```
curl -fsSL https://downloads.poolside.ai/poo
```

Launch and *Log in with Poolside* to get a free API key.

```
pool
```

Use in any ACP client. Configure Zed and JetBrains automatically:

```
pool acp setup --editor zed|jetbrains
```

Use pool with Ollama with one-command setup:

```
ollama pull laguna-xs.2
ollama launch pool --model laguna-xs.2
```

Feedback and issues

Submit feedback with `/feedback` and read the full documentation on GitHub.

Local deployment

Laguna XS.2 is supported in vLLM, SGLang, and Transformers, and TRT-LLM thanks to the support of the team at NVIDIA. Use Laguna-XS.2 with Ollama (with MLX support) and the mlx-lm framework for the best experience on your local machine.

vLLM

Serve Laguna XS.2 locally with vLLM and query it from any OpenAI-compatible client (see Controlling reasoning for tool calls, streaming, and reasoning extraction):

> Laguna XS.2 support is available in vLLM 0.21.0 and later (vllm-project/vllm#41129).

```
pip install 'vllm>=0.21.0'

vllm serve \
    --model poolside/Laguna-XS.2 \
    --tool-call-parser poolside_v1 \
    --reasoning-parser poolside_v1 \
    --enable-auto-tool-choice \
    --served-model-name laguna \
    --default-chat-template-kwargs '{"enable_
```

See the vLLM recipes page for additional deployment guidance.

SGLang

Laguna XS.2 is supported in SGLang via sgl-project/sglang#24204. See the SGLang cookbook entry for a serving recipe.

Speculative decoding (DFlash)

For lower latency, serve Laguna XS.2 with the Laguna-XS.2 DFlash speculator — a 5-layer Llama-style draft model that proposes up to 7 tokens per step at ~70% per-position acceptance on coding tasks.

> DFlash support landed in vLLM via vllm-project/vllm#41880 and is available in vLLM 0.21.0 and later.

```
VLLM_USE_DEEP_GEMM=0 vllm serve poolside/Lag
    --trust-remote-code \
    --enable-auto-tool-choice \
    --tool-call-parser poolside_v1 \
    --reasoning-parser poolside_v1 \
    --speculative-config '{"model":"poolside
```

See the DFlash section of the vLLM recipes page for the full recipe.

Transformers

Laguna XS.2 is supported in Transformers `v5.7.0` and later (huggingface/transformers#45673).

```
import torch
from transformers import AutoModelForCausalLM

model_id = "poolside/Laguna-XS.2"

tokenizer = AutoTokenizer.from_pretrained(mod
model = AutoModelForCausalLM.from_pretrained
    model_id,
    dtype=torch.bfloat16,
    device_map="auto",
)

messages = [
    {"role": "user", "content": "Write a Pyt
]

# Reasoning is on by default; pass enable_th
inputs = tokenizer.apply_chat_template(
    messages,
```

```
    add_generation_prompt=True,
    return_tensors="pt",
    enable_thinking=True,
).to(model.device)

outputs = model.generate(
    inputs,
    max_new_tokens=1024,
    do_sample=True,
    temperature=0.7,
    top_k=20,
)

response = tokenizer.decode(outputs[0][input
print(response)
```

TRT-LLM

> Requires building TensorRT-LLM from the upstream PR that adds Laguna XS.2 support (NVIDIA/TensorRT-LLM#13559). Once that PR merges, the same code will work on a released `tensorrt-llm` wheel.

Laguna XS.2's `configuration_laguna.py` imports a few `transformers >= 4.58` symbols. TRT-LLM currently pins `transformers 4.57`, so the PR ships a `laguna_minimal_overlay.sh` script that symlinks the checkpoint and patches only the config file with a compat shim. Load TRT-LLM against the **overlay directory**, not the original checkpoint.

```
# 1. Check out the PR branch and build TRT-L
git clone https://github.com/NVIDIA/TensorRT
git fetch origin pull/13559/head:laguna && g

# 2. Download the checkpoint.
huggingface-cli download poolside/Laguna-XS.
```

```
# 3. Build the transformers-4.57 compat overl
OVERLAY=$(bash laguna_minimal_overlay.sh ~/mo
```

```
from tensorrt_llm import LLM, SamplingParams

llm = LLM(
    model=OVERLAY,              # overlay pat
    trust_remote_code=True,
    tensor_parallel_size=1,
)

sampling = SamplingParams(max_tokens=1024, te
out = llm.generate(["Write a Python retry wra
print(out[0].outputs[0].text)
```

Or serve with an OpenAI-compatible endpoint:

```
trtllm-serve "$OVERLAY" --port 8000 --trust-r
```

The same recipe works for the FP8 and NVFP4 variants: quantization is detected automatically from `quantization_config`, no extra flags required.

Ollama

Visit Ollama's model library to pull to your local machine.

Controlling reasoning

Laguna XS.2 has native reasoning support and is designed to work best with *preserved thinking*, where `reasoning` content from prior assistant messages is preserved in the message history. This model will generally reason before calling tools and between tool calls.

▶ Expand for example

Disabling reasoning

You can disable thinking by setting `enable_thinking` to `False` in a request or by not providing `--default-chat-template-kwargs {"enable_thinking": True}` or equivalent when starting the server.

▶ Expand for example

For agentic coding use cases, we recommend enabling thinking and preserving reasoning in message history as outlined in the [Controlling reasoning] section.

License

This model is licensed under the [Apache 2.0 License](#).

Intended and Responsible Use

Laguna XS.2 is designed for software engineering and agentic coding use cases, and you are responsible for confirming that it is appropriate for your intended application. Laguna XS.2 is subject to the [Apache 2.0 License](#), and should be used consistently with Poolside's [Acceptable Use Policy](#). We advise against circumventing Laguna XS.2 safety guardrails without implementing substantially equivalent mitigations appropriate for your use case.

Please report security vulnerabilities or safety concerns to security@poolside.ai.